



Savitribai Phule Pune University
Centre For Distance and Online Education

SAVITRIBAI PHULE PUNE UNIVERSITY, PUNE

CENTRE FOR DISTANCE AND ONLINE EDUCATION

Program	MCA	
Course	Programming from First Principles	
Topic Name	Use of first-class functions	
Topic Code	-	
Unit/Module No. & Name	5	Higher Order Functions
Self-Learning Hours	-	

Topic Details

S.No	Topic	Pg.No
1.0	Introduction	5
2.0	Meaning and Definition	6
3.0	First Class Functions	7 - 9
4.0	Lambda Functions	10 - 11
5.0	Iterables And Iterators	12 - 13
6.0	Built-In Higher Order Functions	14
7.0	Map Function	15 - 16
8.0	Filter Function	17 - 18
9.0	Reduce Function	19 - 20
10.0	Any Function	21
11.0	All Function	22

12.0	Advantages Of Higher Order Functions	23
13.0	Keywords Table	24 - 25



OBJECTIVE

1. This self-learning material explains higher order functions in Python in a simple and structured way.
2. It revises first class functions, lambda functions, and iterables before introducing higher order functions. It also explains the built-in higher order functions map, filter, reduce, any, and all.
3. By the end of this lesson, students should be able to define higher order functions, explain how they work with iterables, and use built-in functions to write clear, modular, and reusable Python code.



1.0 INTRODUCTION

In Python, functions are very powerful. They are not only used to perform tasks, but they can also be treated like values. This means they can be stored in variables, passed to other functions, and returned from functions. Because of this property, Python supports higher order functions in a natural and useful way.

Higher order functions help programmers write concise, flexible, and abstract code. Instead of writing long loops again and again, a programmer can apply a function to each element of a collection, select only the required values, or reduce many values into one result. This makes code easier to read and reuse.

Before learning higher order functions, it is helpful to revise three ideas: first class functions, lambda functions, and iterables. These ideas form the base for understanding how higher order functions work in Python. Once these concepts are clear, built-in higher order functions such as map, filter, reduce, any, and all become much easier to understand and use.



Savitribai Phule Pune University
Centre For Distance and Online Education

2.0 MEANING AND DEFINITION

2.1 Meaning Of Higher Order Functions

A higher order function is a function that takes another function as an argument, returns a function, or works with both functions and iterables together. In simple words, it is a function that operates on other functions. This makes it more flexible than an ordinary function.

Higher order functions are useful because they help create common patterns in coding. Instead of repeating the same logic many times, a programmer can use one higher order function with different functions and different iterables. This reduces repetition and improves modularity.

2.1.1 Formal Definition

In academic terms, a higher order function is a function that accepts one or more functions as input, may accept iterables as additional input, and may return either a value, an iterable, or another function. Such functions support abstraction, modularity, and code reuse in programming.

2.2 Main Ideas In The Definition

The definition of higher order functions includes some important ideas. First, a function can act on another function. Second, iterables are often used as data input. Third, the result may be an iterable or a single value. Fourth, higher order functions help write concise and reusable code.

3.0 FIRST CLASS FUNCTIONS

3.1 Meaning Of First Class Functions

Functions in Python are called first class functions when they can be treated in the same way as numbers, strings, and other variables. This means a function is not limited to only being defined and called. It can also be assigned to a variable, passed as an argument to another function, returned from a function, and stored inside data structures such as lists and dictionaries. In simple words, Python treats functions like normal values. This property is very important because it gives programmers more freedom while writing programs. It also makes code more flexible and powerful. Higher order functions depend on this property because they need functions to be passed as input or returned as output. Without first class functions, features such as map, filter, and reduce would not work in the way Python supports them. Therefore, first class functions form the basic foundation of higher order programming and functional style coding in Python.

3.1.1 Main Characteristics Of First Class Functions

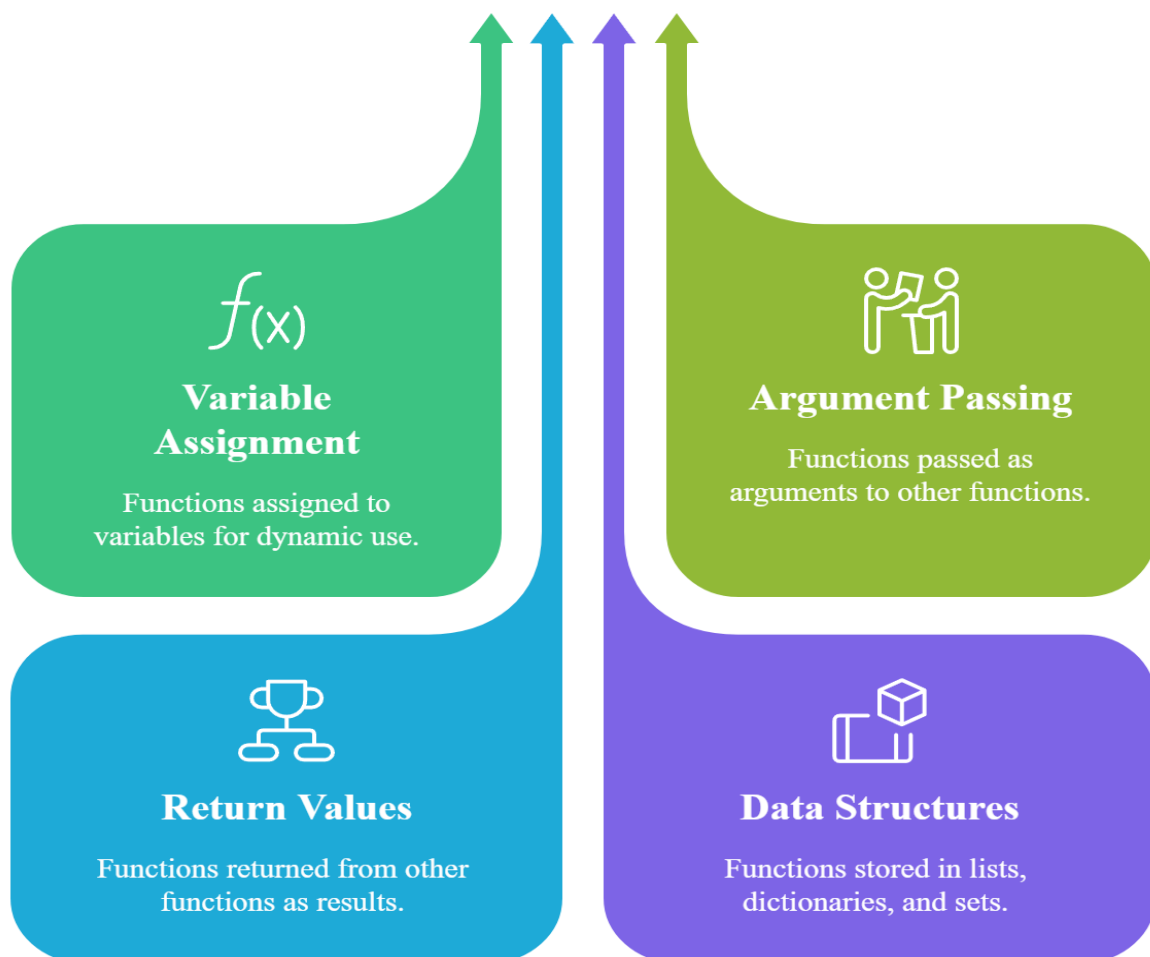
First class functions have some clear characteristics that make them easy to identify in Python. First, a function can be assigned to a variable, just like any other value. Second, a function can be passed as an argument to another function. Third, a function can be returned from another function as a result. Fourth, a function can be stored in collections such as lists, tuples, dictionaries, and sets. These characteristics show that functions are treated as real objects in Python. Because of this, programmers can combine functions in many useful ways. This ability supports abstraction and code reuse. It also allows functions to be handled dynamically during program execution. These characteristics are the reason Python supports a functional programming style along with procedural and object-oriented styles.

3.2 Importance Of First Class Functions

First class functions are important because they make Python more expressive, flexible, and useful for writing modern programs. Since functions can be handled like values, programmers can reuse them easily in many parts of a program. This helps reduce repetition and makes the

code shorter and cleaner. First class functions also support modular programming because one function can be passed to another whenever needed. This improves the organization of the code and makes testing and maintenance easier. They are also the key reason why higher order functions are possible in Python. Built-in functions such as map, filter, and reduce all depend on the ability to treat functions as data. In addition, first class functions support functional programming ideas, where functions become central tools for processing data. Because of this, they help programmers write elegant, reusable, and abstract code in a more efficient and readable way.

Fig. 1 The Power of First Class Functions



This illustration explains first-class functions, showing how functions can be assigned to variables, passed as arguments, returned from other functions, and stored in data structures for flexible programming.

3.2.1 Role In Higher Order Programming

The role of first class functions in higher order programming is very significant. Higher order functions are functions that take other functions as arguments or return functions as output. This is only possible because Python allows functions to behave like values. For example, when we use map, filter, or reduce, we pass a function as an argument. If functions were not first class, such operations would not be possible in the same simple way. This shows that first class functions are not just a language feature, but a practical tool for building advanced programs. They allow programmers to separate logic from data processing and create more generic solutions. As a result, programs become more reusable, more modular, and easier to understand.

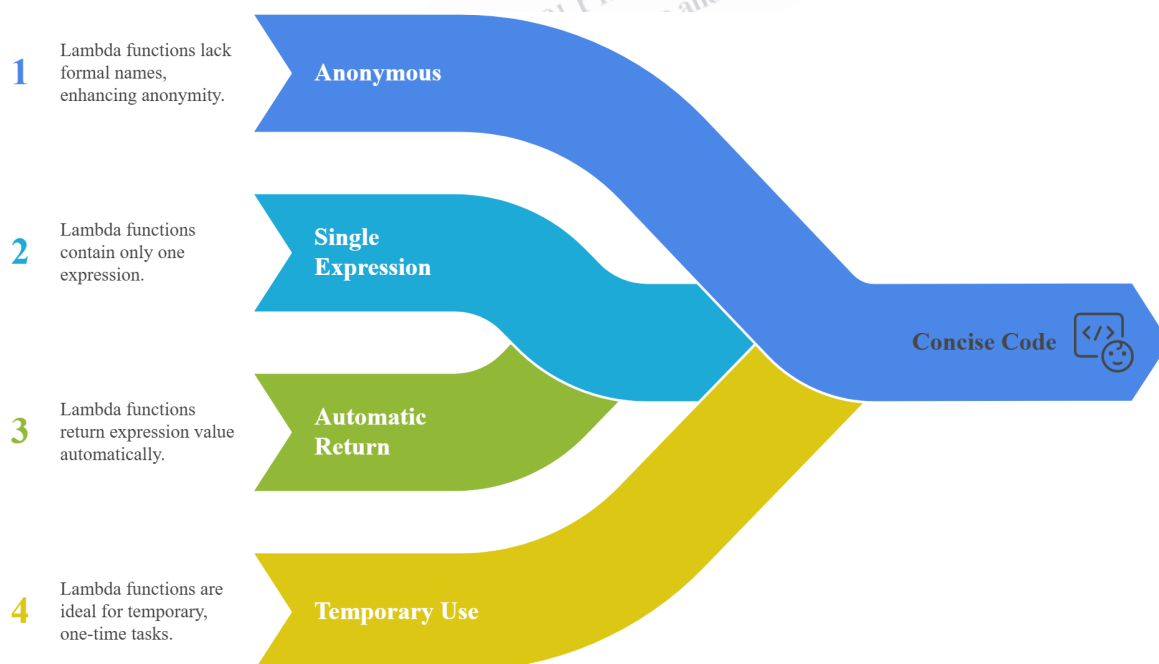


4.0 LAMBDA FUNCTIONS

4.1 Meaning Of Lambda Functions

A lambda function is an anonymous function in Python, which means it does not have a formal name like a function created with the `def` keyword. It is written in a short and simple form and is mainly used when the required logic is small. A lambda function can take any number of arguments, but it can contain only one expression in its body. The value of that expression becomes the return value automatically. This makes lambda functions very compact and useful for quick operations. They are especially helpful when a function is needed only once and does not require a complete separate definition. In such cases, writing a full function may make the code longer than needed. Therefore, lambda functions are often used for temporary tasks, short calculations, and functional programming operations. They are simple, direct, and very useful when small pieces of logic must be written quickly and clearly.

Fig. 2 Lambda Function Advantages



This diagram highlights key advantages of lambda functions, including anonymity, single-expression simplicity, automatic return of values, and suitability for short, temporary tasks, resulting in concise and efficient code.

4.1.1 Features Of Lambda Functions

Lambda functions have some important features that make them different from normal functions. First, they are anonymous, which means they usually do not have a name unless they are assigned to a variable. Second, they are written in a single line, which makes them short and compact. Third, they can accept many arguments, but they are limited to only one expression in the body. Fourth, the result of the expression is returned automatically, so there is no need to write a separate return statement. These features make lambda functions very useful in situations where the logic is simple and temporary. They are often used inside map, filter, and other higher order functions. Because of their short form, they help reduce the size of the code and make simple operations easier to write. However, they are best suited for simple tasks rather than complex logic.

4.2 Example Of Lambda Function

A simple example of a lambda function is checking whether a number is even. In a normal function, we may define a function with def and write a condition that returns true if the number is divisible by 2. The same work can be done more briefly with a lambda expression such as `lambda n: n % 2 == 0`. Here, n is the argument, and the expression checks whether the remainder after division by 2 is equal to 0. If the value 8 is passed, the result becomes true because 8 is divisible by 2 without any remainder. This example shows how lambda functions can replace very small named functions when only one quick operation is needed. It also shows how the syntax stays short while still remaining meaningful. Such expressions are especially useful when used once inside another function, instead of writing a complete separate function definition.

5.0 ITERABLES AND ITERATORS

5.1 Meaning Of Iterables

Iterables are data structures in Python that can store more than one value and allow those values to be accessed one at a time. Common examples of iterables are list, tuple, dictionary, and set. They are called containers because they hold many elements together under one name. An iterable does not show all its values at once in the process of traversal. Instead, it allows the program to move through its elements in sequence. This property makes iterables very useful in loops, data processing, and functional programming. In Python, higher order functions such as map and filter often take iterables as input because they need a collection of elements to work on. Therefore, understanding iterables is important for learning how Python handles groups of values in an organized and efficient way. Iterables provide the base structure from which elements can be read, processed, and transformed one after another.

5.1.1 Importance Of Iterables In Python

Iterables are important in Python because they allow programs to work with collections of data in a simple and systematic way. Instead of storing and processing each value separately, a programmer can place many values inside one iterable object and then handle them together. This makes code shorter, cleaner, and easier to manage. Iterables are widely used in loops, list operations, comprehensions, and higher order functions. Since functions such as map, filter, and reduce depend on a sequence of values, iterables become essential for functional programming in Python. They also help in reading data one element at a time, which is useful when working with large datasets. Because iterables support structured data access, they make Python programming more efficient and more readable. Thus, iterables are not only containers of values, but also important tools for organizing and processing data in many programming tasks.

5.2 Meaning Of Iterators

An iterator is an object that contains a countable number of values and allows those values to be accessed one by one. It is created from an iterable by using the iter() function. Once the iterator is created, its elements are not taken all at once. Instead, individual values are accessed using the

next() function. Each time next() is called, the iterator moves forward and gives the next available value. This process continues until all elements are used. Iterators are important because they make step-by-step traversal possible. They are especially useful when data must be processed in sequence rather than all together. In Python, iterators support efficient memory use because they do not always need to store all results at the same time. They also form the basis of iteration in loops and many built-in functions. Thus, an iterator is a practical tool for controlled and orderly access to the values stored in an iterable.

5.2.1 Example Of Iterator

A simple example of an iterator can be seen with a tuple containing the values Monday, Tuesday, and Wednesday. First, an iterator is created from this tuple by using the iter() function. Suppose the iterator is stored in a variable such as days_it. Now, when the next() function is called with this iterator, it returns Monday as the first element. When next() is called again, it returns Tuesday. A third call to next() returns Wednesday. In this way, the iterator gives access to one element at a time in sequence. This example clearly shows the difference between an iterable and an iterator. The tuple is the iterable because it stores the values, while the iterator is the object used to move through those values. Therefore, iterators provide a clear and efficient way to access items step by step from a collection.

6.0 BUILT-IN HIGHER ORDER FUNCTIONS

6.1 Overview Of Built-In Functions

Python supports several built-in higher order functions that make programming more concise, flexible, and expressive. Important built-in higher order functions include map, filter, reduce, any, and all. These functions work mainly with iterables and help process data in different ways. Some functions apply a transformation to every element, some select only those elements that satisfy a condition, some combine the entire iterable into one final value, and some test whether elements in the iterable are true or false. Because of this, they are widely used in Python for data processing and logical testing. These functions are useful because they reduce code size, remove unnecessary loops, and improve readability. They also support a functional style of problem solving, where functions are used as values and passed to other functions. Therefore, built-in higher order functions are an important part of efficient and elegant Python programming.

6.1.1 Importance Of Built-In Higher Order Functions

Built-in higher order functions are important because they help programmers write shorter, cleaner, and more modular code. Instead of writing long loops for every small task, a programmer can use one built-in function with the required logic and iterable. This saves time and makes the code easier to understand. These functions are also useful because they support abstraction. A programmer can focus on what operation is needed rather than on the detailed steps of iteration. For example, map is used for transformation, filter is used for selection, reduce is used for combining values, and any and all are used for truth checking. This clear division of purpose improves the structure of the program. In addition, these functions are common in Python and are often used in functional programming, data handling, and algorithm design. Thus, they play an important role in writing readable and efficient programs.

7.0 MAP FUNCTION

7.1 Meaning Of Map

The map function is one of the most useful built-in higher order functions in Python. It takes a function as its first argument and an iterable as its second argument. After that, it applies the given function to each element of the iterable one by one. The result is returned as a map object, which can later be converted into a list, tuple, or any other iterable form if needed. In simple words, map helps perform the same operation on every value in a collection. This makes the code shorter and easier to understand. Instead of writing a loop to process each element separately, the programmer can use map to do the work in one clear statement. The map function is especially useful when all elements of a collection need the same kind of transformation. Because of this, it supports concise, readable, and reusable code in Python programming.

7.1.1 Importance Of Map Function

The map function is important because it helps simplify repeated operations on iterable data. When a programmer wants to apply the same logic to every element of a list, tuple, or other iterable, map provides a direct and efficient way to do it. This reduces the need for long loops and temporary variables. It also improves readability because the purpose of the code becomes very clear. Another important benefit of map is that it supports functional programming style in Python. The programmer can write a function separately and then apply it to the full iterable using map. This makes the code more modular and reusable. Since the same function can be used with different collections, it increases flexibility in programming. Therefore, the map function is valuable not only for saving time but also for making programs more elegant, organized, and easier to maintain.

7.2 Example Of Map

Suppose we define a function `square(x)` that returns the square of a number by multiplying `x` with itself. If this function is passed to map along with the list `[2, 4, 6, 8]`, then the map function applies `square` to each value in the list one by one. First, it applies `square` to 2 and gets 4. Then it applies `square` to 4 and gets 16. After that, it applies `square` to 6 and gets 36, and finally it applies

square to 8 and gets 64. The output of map is a map object, but when it is converted into a list, the result becomes [4, 16, 36, 64]. This example clearly shows how map works on every element in the iterable using the same function. It also shows why map is useful for applying one operation to a complete collection quickly and neatly.

7.2.1 Use Of Map With Lambda Function

The map function can also be used with a lambda function when the operation is very small and temporary. In such cases, there is no need to define a separate function with def. For example, instead of writing a square function separately, we can write map with a lambda expression such as `lambda x: x * x`. If this lambda function is used with the list [2, 4, 6, 8], the result will again be [4, 16, 36, 64]. This method is useful when the function logic is short and is not needed elsewhere in the program. It keeps the code compact and direct. The combination of map and lambda is very common in Python because both support functional programming. Thus, using map with lambda makes simple transformations easier to write and helps avoid unnecessary function definitions.



8.0 FILTER FUNCTION

8.1 Meaning Of Filter

The filter function is another important built-in higher order function in Python. It takes two main arguments: a function and an iterable. However, unlike map, it does not apply a transformation to every element. Instead, it checks whether each element satisfies a condition. If the function returns true for a particular element, that element is included in the output. If the function returns false, that element is ignored. Because of this behavior, filter works like a selector. It keeps only the values that match the required condition. The function used with filter should usually return either true or false. Common examples include functions such as is even, is odd, is prime, is positive, or is negative. This makes filter very useful when a programmer wants to extract only specific values from a larger collection. Therefore, the filter function helps in writing short, clear, and efficient code for selecting required elements from iterables.

8.1.1 Importance Of Filter Function

The filter function is important because it makes data selection simple and readable. In many programming tasks, we do not need to process every element in the same way. Sometimes we only want those values that satisfy a given condition. In such situations, filter is very useful because it removes unwanted elements and keeps only the required ones. This reduces the need for long loops and repeated if conditions in the code. It also improves readability because the purpose of the code becomes immediately clear. A reader can easily understand that the program is selecting specific items from a collection. Filter also supports modular programming because the checking condition can be written separately as a function and reused later. When combined with lambda functions, it becomes even more compact. Thus, the filter function is valuable for writing short, neat, and condition-based selection logic in Python programming.

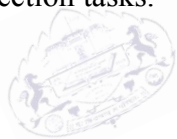
8.2 Example Of Filter

Suppose we define a function `is_negative(n)` that checks whether a number is less than 0. If the number is negative, the function returns true; otherwise, it returns false. Now imagine that we have a list containing both positive and negative numbers. If we pass the function `is_negative`

and that list to filter, Python will examine each element one by one. For every value, it will call `is_negative`. If the answer is true, that number will be included in the output. If the answer is false, that number will be skipped. For example, if the list contains values such as 5, -3, 7, -8, 2, and -4, then the filter function will return only the negative numbers, which are [-3, -8, -4]. This example clearly shows that filter does not change values. It only selects those elements that satisfy the required condition.

8.2.1 Use Of Filter With Lambda Function

The filter function can also be used with a lambda function when the condition is short and simple. In such cases, there is no need to define a separate named function. For example, instead of writing an `is_negative` function with `def`, we can directly write a lambda expression such as `lambda n: n < 0`. If this lambda is used with filter on a mixed list of positive and negative numbers, it will return only the negative values. This makes the code shorter and more direct. The combination of filter and lambda is common in Python because it is useful for quick condition-based selection. It is especially helpful when the logic is small and will be used only once. Therefore, using filter with lambda makes Python code more compact, elegant, and easier to write for simple selection tasks.



9.0 REDUCE FUNCTION

9.1 Meaning Of Reduce

Reduce is a built-in higher order function used to process an iterable and return a single final value. Unlike map and filter, which produce iterable objects as output, reduce keeps combining the elements of the iterable step by step until only one result remains. This is the reason it is called reduce, because it reduces many values into one. In Python, reduce is not directly available like map or filter, so it must be imported from the functools module. The reduce function takes two main arguments: a function and an iterable. The function used with reduce usually takes two parameters and combines them into one result. Then that result is combined with the next element, and the process continues until the iterable is fully processed. This makes reduce very useful when the goal is to calculate a total, product, combined value, or any accumulated result from a collection of elements.

9.1.1 Importance Of Reduce Function

The reduce function is important because it helps convert a whole iterable into one meaningful result. In many programming tasks, the final goal is not to transform every value separately or to filter selected values, but to combine all values into one answer. Examples include finding the sum of numbers, the product of values, the maximum result, or a combined expression. Reduce makes this process systematic and concise. It removes the need to write long loops and repeated update statements. It also supports a functional style of programming, where the combination logic is separated into a function and then applied to the iterable. This improves clarity, reusability, and abstraction in code. Since reduce returns only one final value, it is especially useful in problems where all the values must contribute to a single answer. Thus, reduce is a powerful function for accumulation, aggregation, and step-by-step combination of iterable elements in Python.

9.2 Example Of Reduce

Suppose we define a function `add(x, y)` that returns the addition of two values by writing `x + y`. Now we use reduce on the list `[1, 2, 3, 4, 5]`. The reduce function first takes the first two

elements, 1 and 2, and applies the add function. This gives the result 3. Next, reduce takes this result 3 and combines it with the next element, which is also 3, giving 6. Then it takes 6 and combines it with 4 to get 10. Finally, it takes 10 and combines it with 5 to get 15. At the end of the process, the whole list is reduced to one value, 15. This example clearly shows how reduce works step by step. It combines the values in sequence and produces one final result. Therefore, reduce is very useful when a collection of values must be merged into a single answer.

9.2.1 Use Of Reduce With Lambda Function

The reduce function can also be used with a lambda function when the combining rule is short and simple. In that case, there is no need to define a separate named function such as add. For example, we can write reduce with a lambda expression like `lambda x, y: x + y`. If this is used with the list `[1, 2, 3, 4, 5]`, the result will still be 15. This approach makes the code shorter and more direct, especially when the combining logic is small and used only once. The same method can also be used for multiplication, subtraction, or other binary operations. For example, a lambda can multiply elements or combine them using a custom formula. Thus, using reduce with lambda is helpful when the operation is simple, temporary, and does not require a separate function definition. It keeps the code compact while still maintaining clear logic.

10.0 ANY FUNCTION

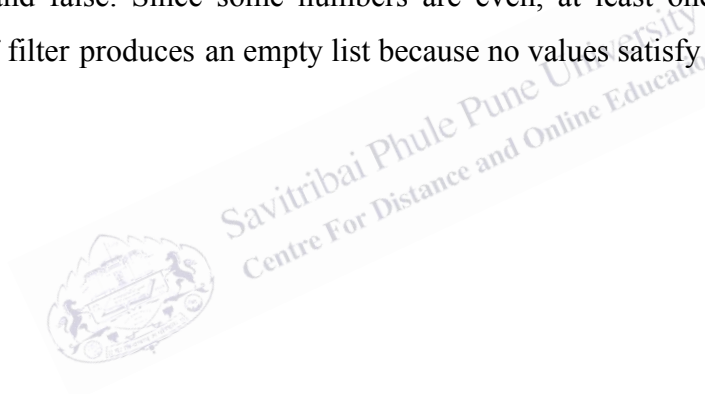
10.1 Meaning Of Any

The any function checks whether at least one element in an iterable is true or truthy. If even one element is true, the result of any is true. If all elements are false, or if the iterable is empty, the result is false.

This function is useful when we want to know whether at least one condition is satisfied in a collection of results. It is simple but very powerful in logical testing.

10.2 Example Of Any

Suppose the output of map with an even-checking function on the list [2, 3, 4, 6, 9] gives values that include true and false. Since some numbers are even, at least one result is true, so any returns true. But if filter produces an empty list because no values satisfy the condition, then any returns false.



11.0 ALL FUNCTION

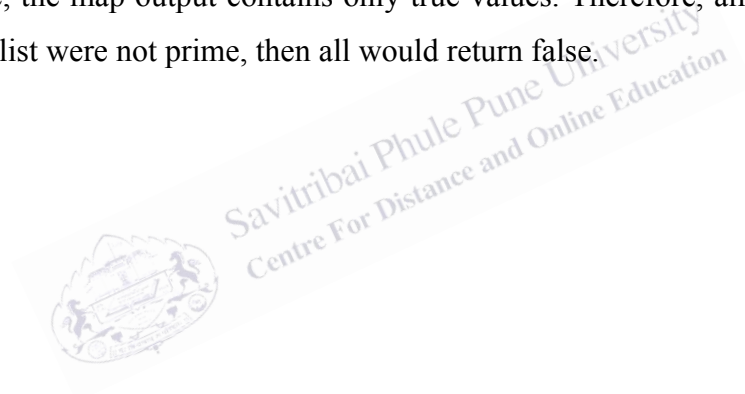
11.1 Meaning Of All

The all function checks whether every element in an iterable is true. It returns true only when all elements are true or truthy. If even one element is false, the result becomes false. Thus, all is stricter than any.

This function is useful when all conditions must be satisfied together. It is often used in validation, testing, and logical checks.

11.2 Example Of All

Suppose we define a prime-checking function and use map on the list [2, 3, 5, 7]. Since all these numbers are prime, the map output contains only true values. Therefore, all returns true. If even one number in the list were not prime, then all would return false.



12.0 ADVANTAGES OF HIGHER ORDER FUNCTIONS

12.1 Major Advantages

Higher order functions offer many benefits. They make code more concise, flexible, and abstract. They help create common coding patterns and reduce repetition. They also support modularity because one function can be reused in many places. This makes programs easier to understand, maintain, and test.

In Python, built-in higher order functions are especially useful because they work naturally with iterables and simple helper functions. This improves both clarity and efficiency in programming.

1. Functions in Python can be treated like values.
2. Lambda functions are anonymous and contain one expression.
3. Iterables include list, tuple, set, and dictionary.
4. Iterators are created with `iter()` and accessed with `next()`.
5. Higher order functions take functions as arguments or return them.
6. Map transforms each element of an iterable.
7. Filter selects only elements that satisfy a condition.
8. Reduce combines all elements into one value.
9. Any returns true if at least one value is true.
10. All returns true only if every value is true.

13.0 KEYWORDS TABLE

Keyword	Touch Vocabulary
First Class Function	A function treated like a value
Lambda Function	An anonymous one-expression function
Iterable	A container that can be traversed
Iterator	An object used to access values one by one
Higher Order Function	A function working on other functions
Map	Applies a function to all elements
Filter	Selects elements that satisfy a condition
Reduce	Combines values into one result
Any	Checks if at least one value is true

All	Checks if every value is true
Functools	Module used to import reduce
Modular Code	Code divided into reusable parts
Reusable Code	Code that can be used again
Abstract Code	Code that hides repeated details
Truthy	A value treated as true in Python

